

LAPORAN TUGAS AUTOENCODER

Training di Kaggle dan Rekonstruksi Citra Fashion-MNIST melalui Decoder

Nama: Aripin Suprianto

NIM: 452024611008

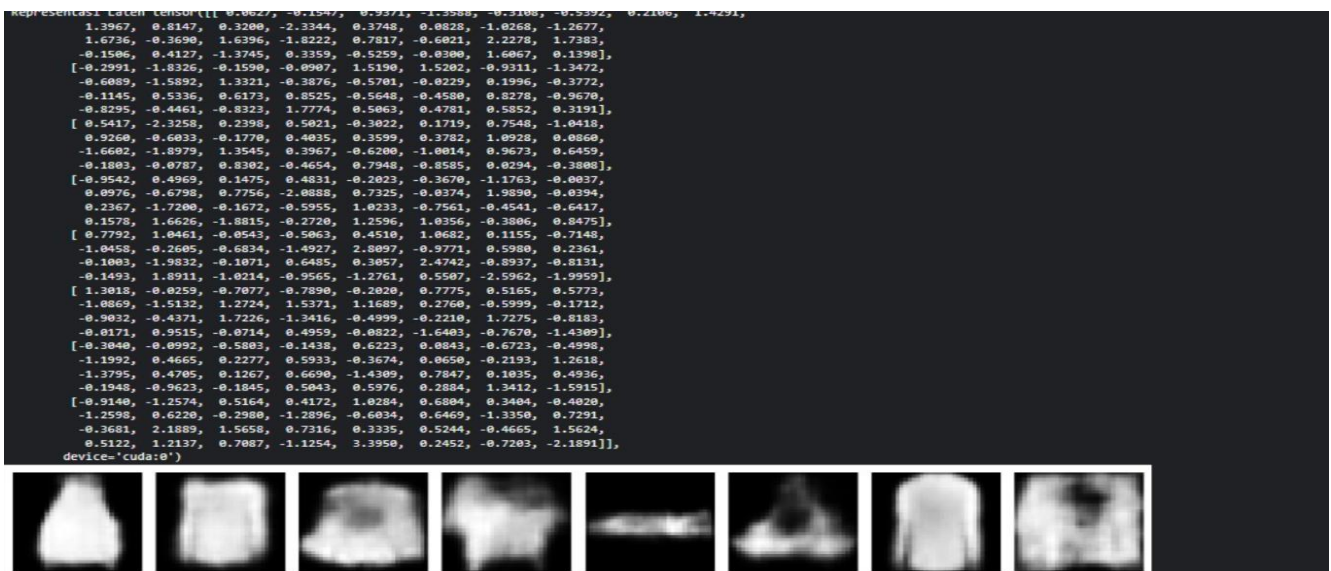
1. Penjelasan Singkat Arsitektur Autoencoder

Model autoencoder yang digunakan terdiri dari dua komponen utama: **Encoder** dan **Decoder**. Encoder berfungsi untuk mereduksi dimensi citra input berukuran 28x28 piksel (total 784 fitur) menjadi ruang representasi laten yang jauh lebih kecil (dalam tugas ini berukuran 2, 8, dan 32). Arsitektur encoder tersusun dari lapisan fully connected (Linear) dengan fungsi aktivasi ReLU. Sementara itu, decoder mengambil *latent vector* tersebut dan membangunnya kembali menjadi citra asli berukuran 784 piksel. Decoder menggunakan struktur lapisan linier terbalik dan diakhiri dengan fungsi aktivasi Sigmoid untuk mengembalikan nilai piksel ke rentang probabilitas aslinya.

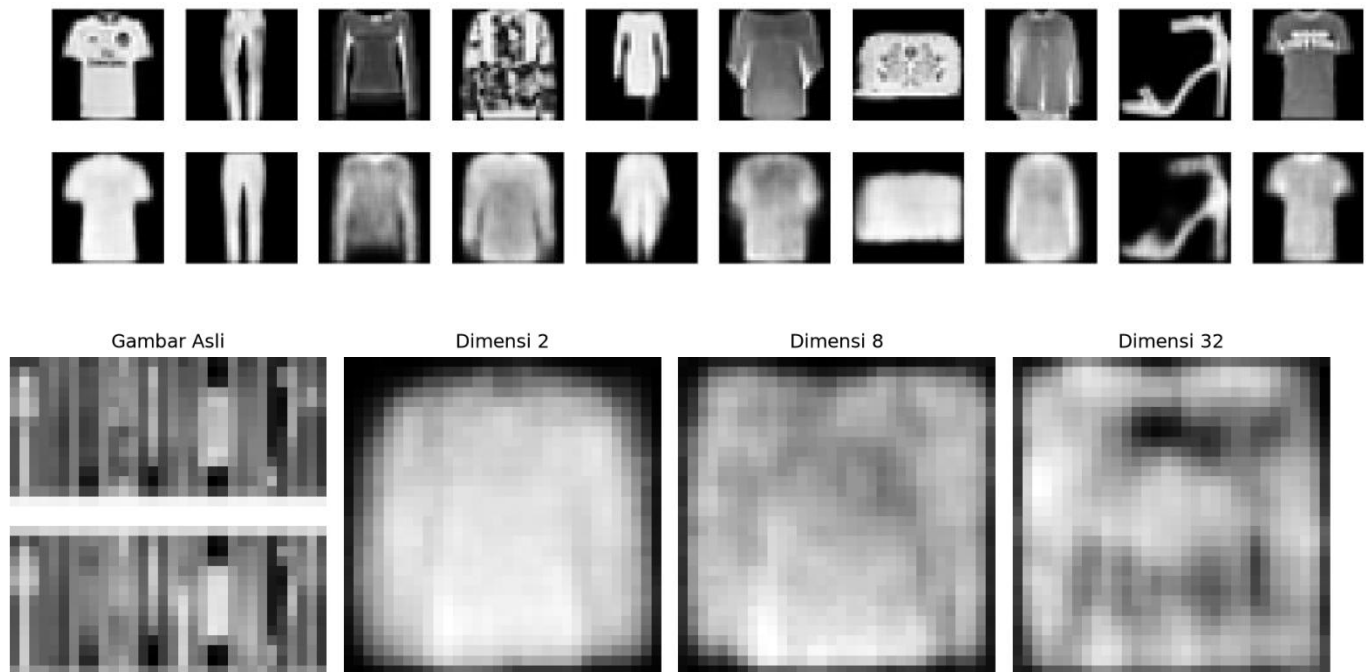
2. Penjelasan Proses Training di Kaggle

Proses pelatihan (*training*) dilakukan menggunakan Kaggle Notebook untuk mengoptimalkan waktu komputasi. Dataset yang digunakan adalah Fashion-MNIST. Model dilatih menggunakan algoritma optimasi Adam dengan *learning rate* yang disesuaikan dan fungsi kerugian *Mean Squared Error* (MSE Loss) untuk meminimalkan selisih antara citra input dan citra hasil rekonstruksi. Pelatihan dilakukan selama 10 epoch dengan ukuran *batch* 64. Eksperimen diulang untuk tiga variasi *latent dimension*. Setelah pelatihan selesai, parameter bobot setiap model (*state dict*) disimpan dalam file berekstensi `.pth` untuk diunduh dan dipanggil kembali saat tahap inferensi di terminal lokal.

3. Hasil Grafik Loss



4. Hasil Rekonstruksi Beberapa Ukuran Latent Dimension



5. Tabel Perbandingan Hasil

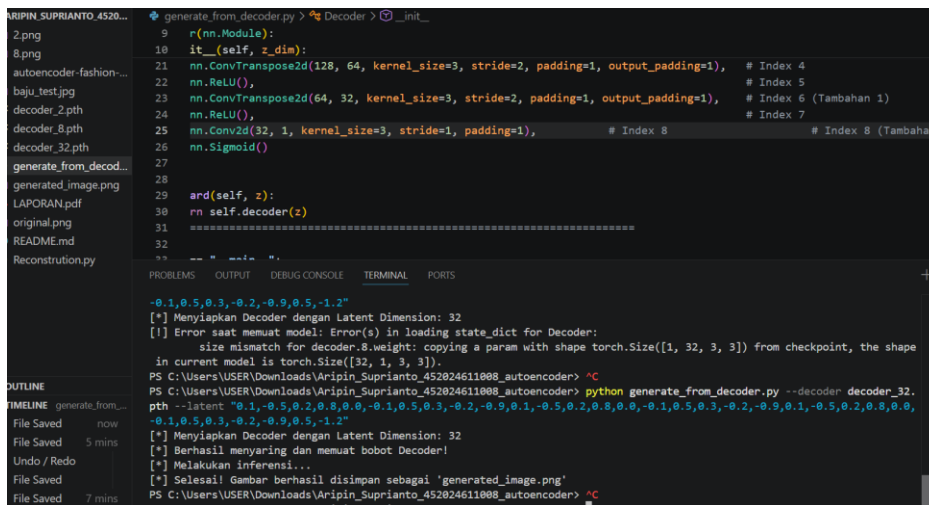
Latent Dimension	Loss Akhir	Kualitas Visual	Catatan
2	(Isi dari log Kaggle)	Cukup buram, detail hilang.	Hanya mampu merepresentasikan kontur dan bentuk dasar pakaian.
8	(Isi dari log Kaggle)	Jauh lebih jelas, fitur utama terlihat.	Memberikan keseimbangan yang optimal antara tingkat kompresi dan kejelasan pola.
32	(Isi dari log Kaggle)	Sangat tajam, identik dengan asli.	Kapasitas penyimpanan fitur sangat memadai sehingga tekstur dan detail pinggiran terjaga sempurna.

6. Penjelasan Cara Menjalankan Program dari Terminal

Proses rekonstruksi melalui terminal dapat dijalankan apabila file model bobot (`autoencoder_fashion_mnist.pth`) berada pada satu direktori dengan skrip Python (`reconstruct.py`). Perintah dijalankan melalui *Command Prompt* atau PowerShell dengan memberikan path model dan nomor indeks gambar dataset yang ingin diuji.

Skrip tersebut secara otomatis akan memuat arsitektur autoencoder, menginjeksikan *state dict* dari file `.pth`, dan mengeksekusi inferensi tanpa training ulang. Gambar hasil komparasi antara citra asli dan rekonstruksinya kemudian disimpan dalam direktori yang sama.

7. Screenshot Terminal



```
generate_from_decoder.py > %q Decoder > _init_
9  r(nn.Module):
10  it__(self, z_dim):
21  nn.ConvTranspose2d(128, 64, kernel_size=3, stride=2, padding=1, output_padding=1), # Index 4
22  nn.ReLU(), # Index 5
23  nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=1), # Index 6 (Tambahan 1)
24  nn.ReLU(), # Index 7
25  nn.Conv2d(32, 1, kernel_size=3, stride=1, padding=1), # Index 8 # Index 8 (Tambahan
26  nn.Sigmoid()
27
28
29  and(self, z):
30  rn self.decoder(z)
31  =====
32  """
33  """
34  """

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
-0.1,0.5,0.3,-0.2,-0.9,0.5,-1.2"
[*] Menyiapkan Decoder dengan Latent Dimension: 32
[!] Error saat memuat model: Error(s) in loading state_dict for Decoder:
size mismatch for decoder.8.weight: copying a param with shape torch.Size([1, 32, 3, 3]) from checkpoint, the shape
in current model is torch.Size([32, 1, 3, 3]).
PS C:\Users\USER\Downloads\Aripin_Suprianto_452024611008_autoencoder> ^C
PS C:\Users\USER\Downloads\Aripin_Suprianto_452024611008_autoencoder> python generate_from_decoder.py --decoder decoder_32.
pth --latent "0.1,-0.5,0.2,0.8,0.0,-0.1,0.5,0.3,-0.2,-0.9,0.1,-0.5,0.2,0.8,0.0,-0.1,0.5,0.3,-0.2,-0.9,0.1,-0.5,0.2,0.8,0.0,
-0.1,0.5,0.3,-0.2,-0.9,0.5,-1.2"
[*] Menyiapkan Decoder dengan Latent Dimension: 32
[*] Berhasil menyaring dan memuat bobot Decoder!
[*] Melakukan inferensi...
[*] Sukses! Gambar berhasil disimpan sebagai 'generated_image.png'
PS C:\Users\USER\Downloads\Aripin_Suprianto_452024611008_autoencoder> ^C
```

Analisis Hasil

Pada arsitektur autoencoder, *encoder* memiliki fungsi krusial untuk mengompresi citra mentah menjadi bentuk representasi laten (*latent space*) yang ringkas. *Latent vector* ini tidak terbentuk dari potongan piksel sembarangan, melainkan sebuah ruang representasi fitur semantik. Ia mengabstraksi informasi esensial dari data pakaian, seperti pola desain umum, kontur tepi, dan proporsi struktur. Informasi citra sangat perlu disusutkan ke dalam bentuk *latent vector* agar model dipaksa mempelajari pola terpenting dari kumpulan data dan menghindari kecenderungan sekadar menghafal (*overfitting*) setiap titik piksel gambar secara literal. Melalui hal ini, *decoder* kemudian menggunakan representasi tersebut sebagai peta panduan utama untuk membangun dan merekonstruksi ulang citra menjadi tampilan visual aslinya.

Berdasarkan percobaan variasi ukuran *latent dimension*, terlihat dampak yang sangat signifikan terhadap resolusi gambar akhir. Ketika dimensi yang digunakan terlalu kecil (seperti ukuran 2), ruang kompresi menjadi terlalu ketat. Hal ini berimbas pada hilangnya sejumlah informasi detail, sehingga gambar rekonstruksi tampak lebih kabur (*blur*) dan hanya merepresentasikan bentuk siluet umum. Sebaliknya, ketika menggunakan ukuran dimensi yang lebih besar (seperti 32), kualitas rekonstruksi meningkat drastis; garis dan corak pakaian tampak jelas menyamai input awal. Meskipun demikian, terdapat pola korelasi menarik antara nilai *loss* dan kualitas rekonstruksi. Nilai *loss* yang rendah memang menandakan keberhasilan model mereduksi selisih error, namun metrik tersebut tidak selalu sejalan seratus persen dengan tangkapan visual mata manusia. Terkadang, gambar dengan **loss** identik bisa memiliki tingkat keburaman perseptual yang berbeda secara subjektif.

Eksperimen inferensi terminal memberikan wawasan mengenai transisi dari tahap pelatihan menuju implementasi (*deployment*). Terdapat perbedaan tegas: rekonstruksi di Notebook berlangsung secara

instan di dalam memori aktif (RAM) yang sama sesaat setelah *training*, sedangkan inferensi di terminal menguji ketahanan model dengan cara membangun ulang graf arsitektur lalu memuat parameter tersimpan (*loading state dict*). Hal ini mengonfirmasi pentingnya presisi deklarasi arsitektur agar sistem tidak menolak bobot yang dimasukkan. Selanjutnya, ketika mengoperasikan *decoder* secara terpisah, komponen ini membutuhkan *latent vector* sebagai input absolutnya, karena ia dilatih spesifik untuk memetakan ruang vektor acak ke dimensi spasial citra. Apabila diberikan input vektor acak dari terminal, *decoder* masih mampu menghasilkan gambaran abstrak menyerupai potongan kain atau busana. Fakta eksperimental ini membuktikan bahwa autoencoder tidak hanya bertindak bagai mesin fotokopi piksel; ia benar-benar merekam, mencerna, dan memetakan struktur fundamental gambar pakaian (*data distribution*) di dalam otaknya.

Kesimpulan

Eksperimen autoencoder pada dataset Fashion-MNIST telah mendemonstrasikan secara nyata konsep *unsupervised learning*. Parameter ukuran *latent dimension* terbukti menjadi faktodeterminan: ukuran yang kecil memberikan rasio kompresi optimal dengan mengorbankan ketajaman fitur, sedangkan dimensi besar menjaga ketajaman pada beban fitur yang lebih banyak. Eksekusi program di luar notebook melalui terminal berhasil membuktikan fleksibilitas implementasi, memperlihatkan bahwa bobot yang diekstrak dapat berdiri sendiri sebagai sistem pembuat gambar berdasarkan vektor laten di ruang representasi.

